Creating Your Own Dataset

Often, the dataset you need to build an **NLP (Natural Language Processing)** application does not already exist. In that case, you need to create the dataset yourself.

In this section, we show how to create a **corpus (a collection of data)** from **GitHub issues**.

On GitHub, issues are commonly used for:

- Reporting bugs
- Suggesting new features
- Asking questions

This type of issue data can be used for several purposes.

What can be done with this dataset

1. **Analyze how long it takes to close an Issue or Pull Request**
2. **Train a multilabel classifier**
   The classifier can automatically assign labels to issues based on their description, such as:
   - bug
   - enhancement
   - question
3. **Build a Semantic Search Engine**
   In this system, when a user enters a query, the system finds the GitHub issues that are most relevant to that query.

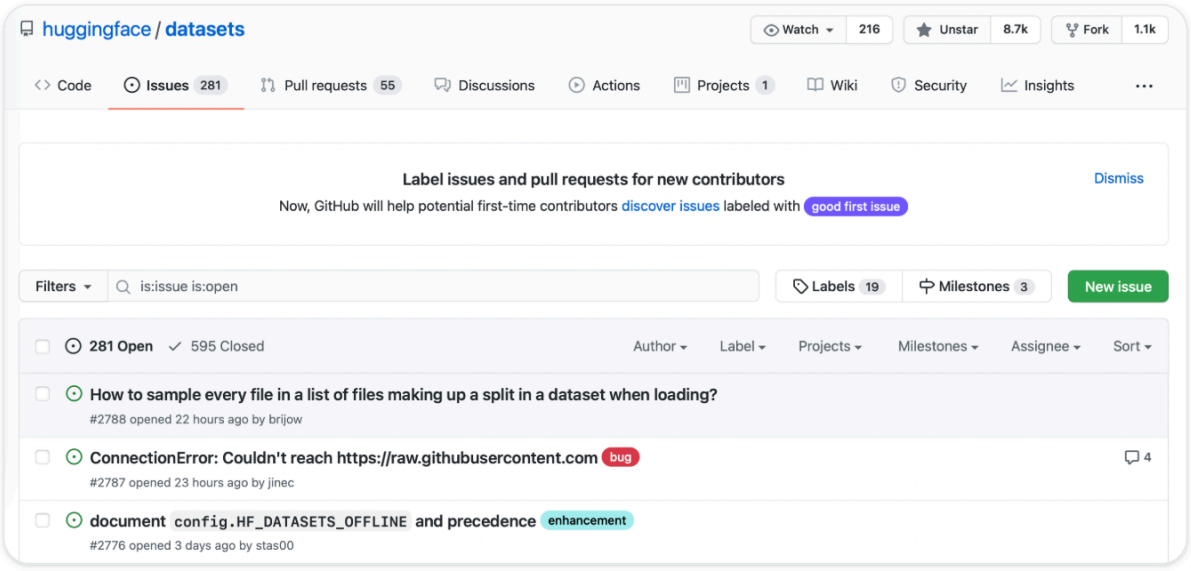
In this section, we focus on **creating the dataset**.
In the next section, we will build a **semantic search application**.

As an example, we will use a popular **open-source project**:

HuggingFace Datasets

5.5.1 Getting the Data

If you open a GitHub repository and go to the **Issues tab**, you can see all the issues.



For example:

- 331 open issues
- 668 closed issues

Each issue usually contains:

- title
- description
- labels
- creator
- comments
- Status

Downloading Issues Using the GitHub API

To download all issues, we will use the:

GitHub REST API

This API returns a **list of JSON objects**, where each object represents one issue.

These objects contain information such as:

- title
- description
- status
- user information
- metadata

Using the GitHub API with Python

We will use Python's **requests library** to send HTTP requests.

Install the library

!pip install requests

Sending an API Request

```
import requests

url = "https://api.github.com/repos/huggingface/datasets/issues?page=1&per_page=1"
response = requests.get(url)
```

This request retrieves the **first issue from the first page**.

HTTP Status Code

```
response.status_code
```

Output:

```
200
```

A status code of **200** means the request was successful.

Viewing the JSON Data

```
response.json()
```

This returns a large **JSON object**.

Some important fields include:

Field	Meaning
title	Title of the issue
body	Description of the issue
number	Issue number
user	The user who opened the issue

state	Whether the issue is open or closed
created_at	When the issue was created
comments	Number of comments

GitHub API Rate Limit

When using the GitHub API, there is a limit.

Without a token

You can only make:

60 requests per hour

With a token

You can make up to:

5000 requests per hour

Using a Personal Access Token

After creating a GitHub token, it should be added to the request header.

GITHUB_TOKEN = xxx

headers = {"Authorization": f"token {GITHUB_TOKEN}"}

Important

You should **never share your GitHub token in a public notebook**.

A better approach is to:

- store it in a `.env` file

- load it using **python-dotenv**

Function to Download All Issues

Next, a Python function is created:

fetch_issues()

This function will:

- call the GitHub API
- download issues in batches
- wait for one hour if the rate limit is reached
- save all data at the end

The data is saved as:

datasets-issues.jsonl

Running the Function

fetch_issues()

Depending on your internet speed, this may take several minutes.

Loading the Dataset

```
issues_dataset = load_dataset(
    "json",
    data_files="datasets-issues.jsonl",
    split="train"
)
```

Output:

```
Dataset({
features: [...],
num_rows: 3019
```

})

This means the dataset contains **3019 rows**.

Why Are There More Issues?

On the GitHub **Issues tab**, you may see around **1000 issues**.

But the dataset contains **3000+ entries**.

This is because:

GitHub's API treats **Pull Requests as Issues**.

So:

- Every **Pull Request is an Issue**
- But not every **Issue is a Pull Request**

How to Identify a Pull Request

In the JSON data, if you see the field:

pull_request

then it means the entry is a **Pull Request**.

5.5.2 Cleaning up the Data

According to the GitHub documentation, we know that the `pull_request` **column** can be used to tell whether an entry is:

- an **Issue**
- or a **Pull Request**

This means the dataset contains both issues and pull requests, so we need to separate them.

Understanding the Difference by Looking at a Random Sample

First, we take some random data from the dataset.

sample = issues_dataset.shuffle(seed=666).select(range(3))

Two things are happening here:

1. `shuffle()`

It randomly shuffles the dataset.

2. `select(range(3))`

It takes only the first 3 samples.

Then the `URL` and `pull_request` columns are viewed together:

```
for url, pr in zip(sample["html_url"], sample["pull_request"]):
    print(f">> URL: {url}")
    print(f">> Pull request: {pr}\n")
```

Here, `zip()` is used to iterate through the two columns together.

Explaining the Output

Example:

URL: https://github.com/huggingface/datasets/pull/850
Pull request: { ... }

Here we can see:

- the URL contains **`pull`**
- the `pull_request` field contains **data**

So, this is a **Pull Request**.

Another example:

URL: https://github.com/huggingface/datasets/issues/2773
Pull request: None

Here:

- the URL contains **`issues`**
- `pull_request = None`

So, this is a normal **Issue**.

Main Difference

`pull_request` field	**Meaning**
`None`	Issue
contains data	Pull Request

Creating a New Column

Now we create a new column in the dataset:

`is_pull_request`

```
issues_dataset = issues_dataset.map(
    lambda x: {"is_pull_request": False if x["pull_request"] is None else True}
)
```

What Does This Do?

This code checks each row.

If:

pull_request == None

then:

is_pull_request = False

That means it is an **issue**.

If it is not `None`:

is_pull_request = True

That means it is a **pull request**.

5.5.3 Cleaning up the Data

From the earlier part of GitHub's documentation, we learned that the **`pull_request`** column can be used to determine whether a record is a normal **issue** or a **pull request**. Now we will look at a few random samples to understand this difference.

Here, we use `Dataset.shuffle()` and `Dataset.select()` to take some random examples from the dataset. Then we look at the `html_url` and `pull_request` columns together so that we can compare the URL and pull request information.

Creating a random sample

sample = issues_dataset.shuffle(seed=666).select(range(3))

```
# Display the URL and pull request information
for url, pr in zip(sample["html_url"], sample["pull_request"]):
    print(f">> URL: {url}")
    print(f">> Pull request: {pr}\n")
```

Example output

```
>> URL: https://github.com/huggingface/datasets/pull/850
>> Pull request: {...}

>> URL: https://github.com/huggingface/datasets/issues/2773
>> Pull request: None

>> URL: https://github.com/huggingface/datasets/pull/783
>> Pull request: {...}
```

What do we see here?

From this output, we can understand two important things:

If it is a Pull Request

- The URL usually contains `/pull/`
- The `pull_request` field contains an object with data

If it is a normal Issue

- The URL contains `/issues/`
- The value of the `pull_request` field is `None`

So, by using the `pull_request` field, we can easily tell which entry is an issue and which one is a pull request.

Creating a new column

Using this difference, we can add a new column to the dataset called **`is_pull_request`**.

```
issues_dataset = issues_dataset.map(
    lambda x: {"is_pull_request": False if x["pull_request"] is None else True}
```

)

What does this code do?

It checks each row:

- If `pull_request` is `None` → `is_pull_request = False`
  (This means it is an **Issue**)
- If `pull_request` is not `None` → `is_pull_request = True`
  (This means it is a **Pull Request**)

In this way, issues and pull requests can be separated easily in the dataset.

A small exercise (Try it out)

Now you are asked to do a small analysis.

Task:

Find the **average time it takes to close an issue** in the Datasets repository.

A few functions may be useful here:

`Dataset.filter()`

This can be used to:

- remove pull requests
- remove issues that are still open

`Dataset.set_format()`

This can be used to convert the dataset into a **DataFrame**.

After that, you can use the `created_at` and `closed_at` times to calculate how long it takes for an issue to be closed.

Bonus Task

Try to calculate the **average time it takes to close a pull request** as well.

Why is the data not being cleaned more right now?

At this stage, we could also:

- remove some unnecessary columns
- rename some columns

But in general, it is a good idea to keep the dataset as **raw (unchanged)** as possible at this stage. This is because the same dataset may later be used in different kinds of applications.

One important thing is still missing from the dataset

There is still an important piece of information missing from the current dataset:

Comments on Issues and Pull Requests

On GitHub, each issue often includes:

- discussion
- questions and answers
- explanations of solutions

All of this can be found in the comments. However, those comments have not yet been added to the dataset.

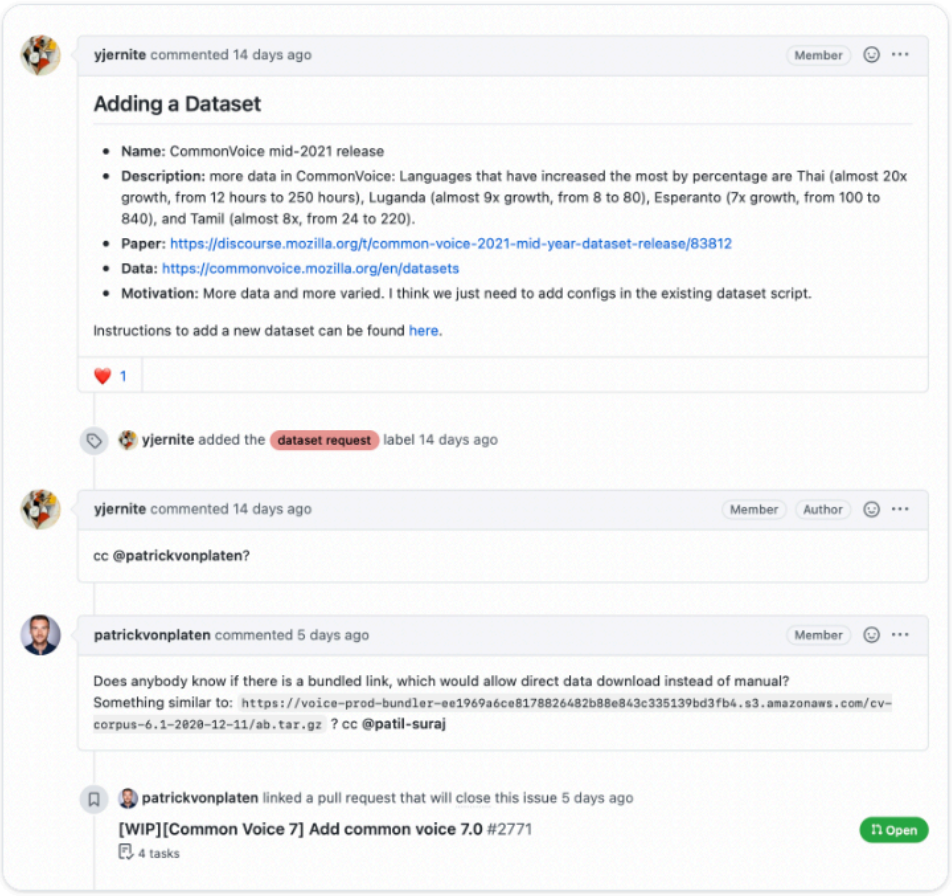
5.5.4 Augmenting the Dataset

In the previous step, we created a dataset from GitHub issues. However, one important piece of information has not yet been added — **comments**.

On GitHub, each issue or pull request usually contains many comments below it. These comments often include:

- detailed discussion about the problem
- suggestions from other developers
- steps for solving the problem
- examples of code

If we want to build a **search engine** or a **question-answering system**, these comments are a very valuable source of information.



Getting Comments Using the GitHub API

The GitHub REST API provides a special endpoint called the **Comments endpoint**.

Using this endpoint, we can retrieve all the comments associated with a specific issue.

For example, suppose the issue number is **2792**.

```
issue_number = 2792
url = f"https://api.github.com/repos/huggingface/datasets/issues/{issue_number}/comments"

response = requests.get(url, headers=headers)
response.json()
```

When this request is sent, the API returns a **JSON list**, where each element represents one comment.

Structure of Comment Data

Each comment returned by the API contains several fields, such as:

Field	Meaning
url	API link
html_url	GitHub comment link
user	The user who wrote the comment
created_at	When the comment was created
body	The actual text of the comment

The most important field is the **body field**, because it contains the main text of the comment.

Example:

@albertvillanova my tests are failing here...
AssertionError: False is not true
Any suggestions on how can I avoid this error?

Here, a developer is asking a question about an error.

Writing a Function to Get Comments

Now we create a small function that retrieves all the comments for a given issue.

```
def get_comments(issue_number):
    url = f"https://api.github.com/repos/huggingface/datasets/issues/{issue_number}/comments"
    response = requests.get(url, headers=headers)
    return [r["body"] for r in response.json()]
```

What this function does

1. Sends a request to the comments endpoint for the given issue
2. Receives the JSON response from the API
3. Extracts the **body text** from each comment
4. Returns all comment texts as a list

Testing the Function

get_comments(2792)

Output example:

["@albertvillanova my tests are failing here...
AssertionError: False is not true ..."]

This means we successfully retrieved all the comment text for that issue.

Adding Comments to the Dataset

Now we add comments to each issue in the dataset.

```
issues_with_comments_dataset = issues_dataset.map(
    lambda x: {"comments": get_comments(x["number"])}
)
```

What happens here

For each row in the dataset:

- `x["number"]` → issue number
- `get_comments()` → retrieves comments for that issue
- a new column called **comments** is added

As a result, the dataset becomes richer and more informative.

Structure of the New Dataset

The dataset now contains columns such as:

Column	Meaning
title	Issue title

body	Issue description
labels	Issue labels
state	Open or closed
number	Issue number
is_pull_request	Whether it is a pull request
comments	All comments for that issue

Why Comments Are Important

Adding comments makes the dataset more powerful because:

- discussions about the issue become available
- explanations of errors can be found
- user problems are easier to understand
- semantic search works better

For example, if a user searches for:

dataset loading error

the system can match not only the **issue title**, but also the **comments**.

5.5.5 Uploading the Dataset to the Hugging Face Hub

Now that our enriched dataset is ready, the next step is to **upload it to the Hugging Face Hub** so that others can use it.

Uploading a dataset is very simple. Similar to uploading models or tokenizers in **Transformers**, we can use the `push_to_hub()` method for datasets.

Logging In First

Before uploading the dataset, you need to log in with your **Hugging Face account**.

If you are working inside a notebook, use:

from huggingface_hub import notebook_login

notebook_login()

What this does

Running this command opens a **login widget** where you can enter:

- your username
- password or API token

The authentication token will then be saved at:

~/.huggingface/token

This means you only need to log in once.

Logging in from the Terminal

If you are working in a terminal instead of a notebook, you can log in using the CLI:

huggingface-cli login

You can also enter your token here.

Uploading the Dataset

After logging in, you can upload the dataset easily:

issues_with_comments_dataset.push_to_hub("github-issues")

What this line means

- `issues_with_comments_dataset` → the dataset you created
- `"github-issues"` → the name of the dataset repository on the Hub

This command uploads the dataset to the Hugging Face Hub.

How Others Can Use the Dataset

Once the dataset is uploaded, anyone can download it using `load_dataset()`.

remote_dataset = load_dataset("lewtun/github-issues", split="train")
remote_dataset

What happens here

- `"lewtun/github-issues"` is the dataset repository ID
- `split="train"` loads the train split of the dataset

The output shows the dataset structure, for example:

Dataset({
    features: [...],
    num_rows: 2855
})

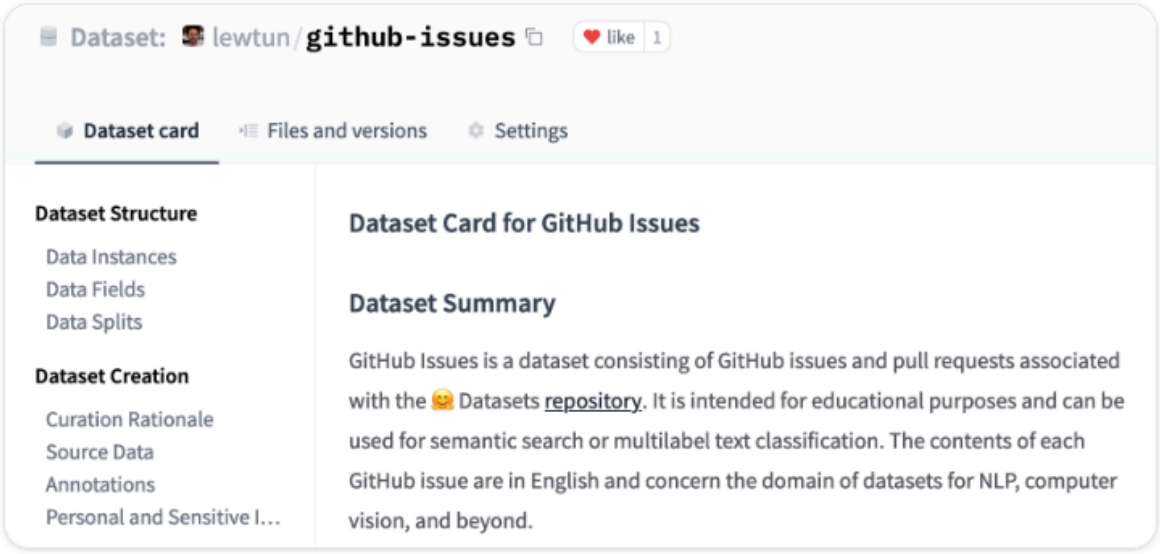
This means the dataset contains **2855 rows**.

5.5.6 Creating a Dataset Card

A well-documented dataset is much more useful for others—and even for your future self. By reading the **dataset card**, a user can understand:

- what the dataset is about
- whether it is suitable for their task
- whether the dataset may contain any bias
- whether there are any risks associated with using the dataset

In other words, simply uploading a dataset is not enough. You should also clearly describe its **purpose, source, usage, and limitations**.



Where is this information stored on the Hugging Face Hub?

On the Hugging Face Hub, this information is stored in each dataset repository's:

`README.md` file

This means a dataset card is essentially a **README file** that contains detailed information about the dataset.

What should be done before creating a Dataset Card?

Before creating this file, two important steps should usually be followed.

1) Creating Metadata Tags

The first step is to create **metadata tags**.

These tags are usually written in **YAML format**.

They are used in various search features on the Hugging Face Hub, which helps members of the community find your dataset more easily.

Why is this necessary?

For example, someone might search for:

- NLP dataset
- issue tracking dataset
- English text dataset
- classification dataset

If your dataset has proper metadata tags, it will appear more easily in search results.

What should be used for this?

You need to use the:

datasets-tagging application

Since we created a **custom dataset**, you need to clone the `datasets-tagging` repository and run the application locally.

In other words, you will use this tool to generate the correct tags for your dataset.

2) Following the Guide for Writing an Informative Dataset Card

The second step is to read the 🤗 **Datasets guide**.

This guide explains how to write a good dataset card.

You can use it as a **template** to complete the `README.md` file.

A dataset card usually includes information such as:

- what the dataset is
- where the data comes from
- how the dataset was created
- the structure of the dataset
- possible use cases

- limitations of the dataset
- potential bias or risks

Where should the README.md file be written?

You can create the **README.md** file directly inside the **Hugging Face Hub**.

There is also an example dataset card available in the repository:

`lewtun/github-issues`

This repository contains a template dataset card that can help you create your own dataset card more easily.

Exercise

You are asked to:

Use the **datasets-tagging application** and the 🤗 **Datasets guide** to complete the **README.md** file for your GitHub issues dataset.

In other words, your tasks are:

- create metadata tags for the dataset
- write the dataset card
- complete the `README.md` file.